

Exact Approaches for the Knapsack Problem with Setups

Fabio Furini*

Michele Monaci[†]

Emiliano Traversi[‡]

Abstract

We consider a generalization of the knapsack problem in which items are partitioned into classes, each characterized by a fixed cost and capacity. We study three alternative Integer Linear Programming formulations. For each formulation, we design an efficient algorithm to compute the linear programming relaxation (one of which is based on Column Generation techniques). We theoretically compare the strength of the relaxations and derive specific results for a relevant case arising in benchmark instances from the literature. Finally, we embed the algorithms above into a unified implicit enumeration scheme which is run in parallel with an improved Dynamic Programming algorithm to effectively solve the problem to proven optimality. An extensive computational analysis shows that our new exact algorithm is capable of efficiently solving all the instances of the literature and turns out to be the best algorithm for instances with a low number of classes.

Keywords: Knapsack Problems , Column Generation , Relaxations , Branch-and-Bound Algorithms , Computational Experiments.

1 Introduction

The classical *Knapsack Problem* (KP) is one of the most famous problems in combinatorial optimization. Given a knapsack capacity C and a set $N = \{1, \dots, n\}$ of items, the j -th having a profit p_j and a weight w_j , KP asks for a maximum profit subset of items whose total weight does not exceed the capacity. KP can be formulated using the following Integer Linear Program (ILP):

$$\max \left\{ \sum_{j \in N} p_j x_j : \sum_{j \in N} w_j x_j \leq C, x_j \in \{0, 1\}, j \in N \right\} \quad (1)$$

where each variable x_j takes value 1 if and only if item j is inserted in the knapsack.

KP is NP-hard, although in practice fairly large instances can be solved to optimality within low running time. The reader is referred to [17, 13] for comprehensive surveys on applications and variants of this problem.

*Université Paris Dauphine, PSL Research University, LAMSADE, 75016 Paris, France
fabio.furini@dauphine.fr

[†]DEI, University of Bologna, 40136 Bologna, Italy
michele.monaci@unibo.it

[‡]Laboratoire d'Informatique de Paris Nord, Université de Paris 13, 93430 Villetaneuse, France,
emiliano.traversi@lipn.univ-paris13.fr

In this paper we consider a generalization of KP arising when items are associated with operations that require some setup time to be performed. In particular, there is a given set $I = \{1, \dots, m\}$ of *classes* associated with items, and each item j belongs to a given class $t_j \in I$. A positive *setup cost* f_i is incurred and a positive *setup capacity* s_i is consumed in case items of class i are selected in the solution. Without loss of generality, we assume that all input parameters have integer values. The resulting problem is known in the literature as Knapsack Problem with Setup (KPS).

KPS has been first introduced in the literature by [15] in a survey of non-standard knapsack problems worthy of investigation. In particular, this variant of KP was listed as it finds many practical application, e.g., when industries that produce several types of products must prepare some machinery related to the production of a certain class of products. In addition, it appears as a subproblem in scheduling capacitated machines, and may be used to model resource allocation problems. [10] designed a Lagrangean Decomposition for the setup knapsack problem, that may be seen as a variant of KPS in which the setup cost of each class and the profit associated to each item can take also negative values. The version of the problem in which only the setup cost for each class is taken into account, usually denoted as fixed charge knapsack problem, was addressed by [1] and [2]. In particular, the former presents an exact algorithm based on a branch-and-bound scheme, while the latter uses cross decomposition to solve the case in which items can be taken at a fractional level. The problem addressed by [19] is the multiple-class integer knapsack problem, a special case of KPS in which item weights are assumed to be a multiple of their class weight, and lower and upper bounds on the total weight of the used classes are imposed. For this problem, different ILP formulations were introduced and an effective branch-and-bound algorithm was designed. A Branch-and-Bound algorithm for KPS was given in [22]. This algorithm was tested on instances with up to 10.000 variables, and turned out to be effective mainly for instances where profits and weight are uncorrelated – while it ran out of memory for several large correlated instances. Motivated by an industrial application in a packing industry, KPS was studied by [5]; this article presented a basic dynamic programming scheme and an improved version of the algorithm, with a reduced storage requirement, that proved able to solve instances with up to 10000 items and 30 classes. Recently, KPS has also been addressed in [21] and [6]. The former introduces a new dynamic programming algorithm, gives negative results on the approximability of the problem in the general case, and considers some special cases for which fully polynomial time approximation schemes exist. The latter presents an exact approach for KPS based on the solution of several ILP models that turn out to be easy to solve in practice. Computational experiments reported in [6], both on instances from the literature and on a large set of new randomly generated problems, show that, for many classes of problems, this approach is the state-of-the-art for the exact solution of KPS. For what concerns approximate solutions, we mention a recent paper [14], where a tree search combination heuristic is presented. The algorithm is based on the definition of a truncated tree search, where at each level only potentially good nodes are candidates for further exploration, and is tested on the instances introduced in [5].

Paper Contributions The contribution of the paper is twofold, as it embraces both theoretical

and computational aspects. We develop linear-time algorithms for the optimal solution of the Linear Programming (LP) relaxation of two Integer Linear Programming formulations of KPS. Computational experiments show that these algorithms produce a considerable speedup with respect to the direct use of a commercial LP solver. In addition, we derive for the first time an effective column generation approach to solve a KPS formulation with a pseudo-polynomial number of variables. Finally, we exploit these fast and strong relaxations within an unified branch-and-bound(-and-price) scheme. By reducing the space complexity of the Dynamic Programming algorithm proposed in [5], we managed to improve its computational performance. Since the new exact algorithms are particularly effective on complementary subsets of KPS instances, in order to obtain the best computational performance, we propose a parallel algorithm which exploits the qualities of all the new exact algorithms. We test our new exact algorithms on a large set of instances proposed in the literature and on a new set of larger randomly generated problems. The outcome of our experiments is that the new approaches are competitive with the state-of-the-art exact algorithms for KPS, though they do not require the use of an ILP solver. In addition, we show that on some classes of instances, a considerable speedup may be obtained with respect to the other algorithms proposed so far in the literature.

In the rest of the paper we will denote by n_i the number of items in each class $i \in I$. We assume that $n_i \geq 2$ for some class $i \in I$ and $m > 1$; otherwise, one could associate the setup capacity and cost to the items, yielding a KP. Without loss of generality, we assume that items are sorted according to their class, i.e., class i includes all items $j \in K_i := [\alpha_i, \beta_i]$, where $\alpha_i = \sum_{k=1}^{i-1} n_k + 1$ and $\beta_i = \alpha_i + n_i - 1$. Moreover, we assume that, within each class, items are sorted according to non-increasing profit over weight ratio, i.e.,

$$\frac{p_j}{w_j} \geq \frac{p_{j+1}}{w_{j+1}} \quad j = \alpha_i, \dots, \beta_i - 1; \quad i \in I.$$

To avoid pathological situations, we also assume that the cost of each class $i \in I$ is smaller than the total profit of its items, i.e., $f_i < \sum_{j \in K_i} p_j$, since otherwise this class will never be used in any optimal solution. We assume that not all items (and classes) can be selected, i.e., $\sum_{j \in J} w_j + \sum_{i \in I} s_i > C$; otherwise a trivial optimal solution is obtained by taking all items and classes. Finally, we assume that each item $j \in N$ satisfies $w_j + s_{t_j} \leq C$; otherwise item j cannot be inserted in any feasible solution, and can be removed from consideration.

Let us introduce a first numerical example, called *Example 1* and reported in Figure 1. The optimal solution value of Example 1 is 132 and the corresponding solution takes both items of the second class. This example will be used to demonstrate some properties of the KPS models in the following sections.

- $m = 2, n = 4, C = 152$;
- $n_1 = 2, \alpha_1 = 1, \beta_1 = 2, n_2 = 2, \alpha_2 = 3, \beta_2 = 4$;
- $f_1 = 10, s_1 = 10, f_2 = 9, s_2 = 6$;
- $p_1 = 84, w_1 = 75, p_2 = 75, w_2 = 72, p_3 = 70, w_3 = 64, p_4 = 71, w_4 = 78$.

Figure 1: Example 1

The paper is organized as follows: in Section 2 we introduce alternative formulations of KPS and discuss the properties of the associated linear programming relaxations. In Section 3 we give efficient combinatorial algorithms for solving the LP relaxations of the models; these algorithms are embedded into an enumerative algorithm described in Section 4. In Section 5 we discuss some improvements to the dynamic programming algorithm proposed in [5]. Section 6 describes a relevant special case of KPS and shows the additional properties of the models in this case. Finally, Section 7 reports an extensive computational experience on the solution of the ILP models (and their relaxations) using our algorithms, and compares their performance with other approaches from the literature, and Section 8 draws some conclusions.

2 Integer Linear Programming models for KPS

In this section we introduce alternative formulations for KPS and discuss the relation between the associated linear programming relaxations. These formulations and the associated LP relaxations will be computationally tested in Section 7.

2.1 Model M1

A natural model for KPS is obtained by introducing x_j variables that have the same meaning as in (1), and decision variables y associated with item classes: in particular, each variable y_i takes value 1 if and only if some item of class i is included in the solution. The resulting model is as follows

$$\max \quad \sum_{j \in N} p_j x_j - \sum_{i \in I} f_i y_i \quad (2)$$

$$\sum_{j \in N} w_j x_j + \sum_{i \in I} s_i y_i \leq C \quad (3)$$

$$x_j \leq y_{t_j} \quad j \in N \quad (4)$$

$$x_j \in \{0, 1\} \quad j \in N \quad (5)$$

$$y_i \in \{0, 1\} \quad i \in I. \quad (6)$$

The objective function (2) maximizes the total profit of the selected items minus the setup cost of the used classes, whereas constraint (3) takes into account that the sum of the item weights and the class setups must not exceed the capacity. Inequalities (4) force a class to be used whenever some item of the class is selected. Finally, (5)–(6) impose all variables to be binary. It is worth mentioning that constraints (4)–(5) and the objective function force the y variables to be binary; thus, in principle, constraints (6) are redundant. The resulting model, denoted as M1 in the following, has $n + m$ variables and $n + 1$ constraints, plus variable domain constraints.

By replacing constraints (5)–(6) with the following ones:

$$x_j \in [0, 1] \quad j \in N \quad (7)$$

$$y_i \in [0, 1] \quad i \in I \quad (8)$$

we obtain the LP relaxation of M1. We will denote by LP1 this relaxation and by U^1 the associated upper bound value. An effective combinatorial algorithm to solve LP1 is given in Section 3.

2.2 Model M2

In this section we present a lighter model, that contains fewer constraints than M1, obtained by replacing constraints (4) with the following ones

$$\sum_{j \in K_i} \bar{w}_j x_j \leq C_i^{\bar{w}} y_i \quad i \in I. \quad (9)$$

Constraints (9) link x and y variables and represent a surrogate relaxation of constraints (4), using non-negative *surrogate weights* $\bar{w}_j$ ($j \in N$). For each class $i \in I$ coefficient $C_i^{\bar{w}}$ can be defined as follow

$$C_i^{\bar{w}} = \max \left\{ \sum_{j \in K_i} \bar{w}_j \theta_j : \sum_{j \in K_i} w_j \theta_j \leq C - s_i, \theta_j \in \{0, 1\}, \forall j \in K_i \right\}, \quad (10)$$

i.e., it can be computed solving a KP with profits and weights defined by the surrogate and original weights, respectively. The capacity of this KP can be set to $C - s_i$ in order to take into account the setup capacity, if some item of class i is selected.

The mathematical model defined by (2)-(3)-(5)-(6) and (9) will be denoted as M2, and corresponds to a family of valid formulations for KPS, defined according to weights $\bar{w}$.

The first natural choice for $\bar{w}$ is to use the original item weights, obtaining the following surrogate constraints:

$$\sum_{j \in K_i} w_j x_j \leq C_i^{\bar{w}} y_i \quad i \in I. \quad (11)$$

A second alternative is to use unitary surrogate weights $\bar{w}$:

$$\sum_{j \in K_i} x_j \leq C_i^{\bar{w}} y_i \quad i \in I. \quad (12)$$

Summarizing, we considered 2 different variants of model M2, obtained by using different values of $\bar{w}_j$ for each item $j \in N$, namely:

- M2_A: $\bar{w}_j = w_j$, i.e., reducing constraints (9) to (11);
- M2_B: $\bar{w}_j = 1$, i.e., reducing constraints (9) to (12).

The formulation above has the same number of variables as M1, but only $m + 1$ constraints (instead of $n + 1$). We will denote by LP2 the linear programming relaxation of model M2, i.e., the problem defined by (2), (3), (9), (7) and (8). In addition, let U_A^2 and U_B^2 be the upper bound

values associated with the LP relaxations of M2_A and M2_B, respectively. The following result shows that there is no dominance between the bounds given by the linear programming relaxations of the models above and LP1.

Proposition 1. *There is no dominance between bounds U^1 , U_A^2 and U_B^2 .*

Proof. We show the thesis by giving two numerical instances showing that there is no dominance between the three bounds. Consider the instance of Example 1. An optimal solution for LP1 is given by $x_1^* = x_2^* = y_1^* = 0.968153$ and has value $U^1 = 144.254$. Model M2_A has $C_1^{\bar{w}} = 75$ and $C_2^{\bar{w}} = 142$. An optimal solution of its LP relaxation is $x_1^* = x_3^* = 1, x_4^* = 0.003638$ and $y_1^* = 1, y_2^* = 0.452703$, yielding an upper bound $U_A^2 = 140.183$. Model M2_B has $C_1^{\bar{w}} = 1$ and $C_2^{\bar{w}} = 2$, and an optimal solution of its LP relaxation is $x_1^* = x_3^* = 1$ and $y_1^* = 1, y_2^* = 0.5$, with value $U_B^2 = 139.5$.

Conversely, consider the following instance with $m = 2$ classes. Class 1 has $n_1 = 2, f_1 = 20$ and $s_1 = 1$, while class 2 has $n_2 = 2, f_2 = 1$ and $s_2 = 1$, and the capacity is 42. The first item has $p_1 = 100$ and $w_1 = 30$, while all the remaining 3 items have profit 10 and weight 10. An optimal solution for LP1 is given by $x_1^* = x_2^* = y_1^* = 1$ and $x_3^* = x_4^* = y_2^* = 0.047$ and has value $U^1 = 90.904$. Model M2_A has $C_1^{\bar{w}} = 40$ and $C_2^{\bar{w}} = 20$. An optimal solution of its LP relaxation is $x_1^* = x_3^* = 1, x_4^* = 0.071$ and $y_1^* = 0.75$ and $y_2^* = 0.535$, yielding an upper bound $U_A^2 = 95.178$. Model M2_B has $C_1^{\bar{w}} = C_2^{\bar{w}} = 2$, and an optimal solution of its LP relaxation is $x_1^* = x_3^* = 1, x_4^* = 0.095$ and $y_1^* = 0.5, y_2^* = 0.547$, with value $U_B^2 = 100.404$. $\square$

Observe that the result above is valid in case coefficients $C_i^{\bar{w}}$ in M2 are computed according to (10), which requires the solution of a KP.

2.3 Model M3

In this section we present an extended model which contains an exponential number of variables. Let us introduce the following collections $\mathcal{S}_i$ ($i \in I$) of feasible subsets of items $S \subseteq K_i$ satisfying the knapsack capacity C

$$\mathcal{S}_i = \left\{ S \subseteq K_i : \sum_{j \in S} w_j \leq C - s_i \right\}. \quad (13)$$

For each item subset $S \in \mathcal{S}_i$, we can define its profit and weight taking also into account the setup cost and capacity of the corresponding class $i(S)$:

$$P_S = \sum_{j \in S} p_j - f_{i(S)}, \quad W_S = \sum_{j \in S} w_j + s_{i(S)}.$$

A valid model for KPS can be obtained by introducing, for each subset $S \in \mathcal{S}_i$ ($i \in I$), a binary variable ξ_S which takes value 1 if and only if subset S is included in the solution:

$$\max \quad \sum_{i \in I} \sum_{S \in \mathcal{S}_i} P_S \xi_S \quad (14)$$

$$\sum_{i \in I} \sum_{S \in \mathcal{S}_i} W_S \xi_S \leq C \quad (15)$$

$$\sum_{S \in \mathcal{S}_i} \xi_S \leq 1 \quad i \in I \quad (16)$$

$$\xi_S \in \{0, 1\} \quad i \in I, S \in \mathcal{S}_i. \quad (17)$$

Objective function (14) maximizes the total profit of the selected subset of items, whereas constraint (15) ensures that the solution satisfies the capacity constraint. Inequalities (16) impose that at most one subset is selected for each class, whereas constraints (17) impose all variables be binary. The resulting formulation, denoted as M3 in the following, corresponds to the classical formulation of the Multiple-Choice Knapsack Problem (MCKP) with inequality constraints; see [12].

By replacing constraints (17) with the following ones:

$$\xi_S \geq 0 \quad i \in I, S \in \mathcal{S}_i, \quad (18)$$

we obtain the linear programming relaxation of M3, that will be denoted as LP3 in what follows. Let U^3 be the associated upper bound value. Note that constraints (16) implicitly provide an upper bound of value 1 on the ξ_{S_i} variables, thus we do not need to impose this bound in (18).

Finally, we observe that the model above has already been proposed by [4] and used, e.g., by [19] to derive an exact approach. In addition, in both papers above, the authors observed that the set of variables in the model can be reduced to a pseudo-polynomial number, considering at most one variable for each item class and possible value of capacity. Since this would require a huge number of variables in large instances, in our approach we used the model above by generating variables on-the-fly, according to a column generation scheme (see Section 3.3).

The quality of the upper bound U^3 cannot be worse than its counterpart associated with all the other models.

Proposition 2. *Bound U^3 dominates bounds U^1, U_A^2 and U_B^2 .*

Proof. To prove the result, it is enough to consider the model, say $M1^+$, obtained by adding to M1 the following valid constraints:

$$\sum_{j \in K_i} w_j x_j \leq C - s_i y_i, \quad i \in I. \quad (19)$$

We first observe that these inequalities are implied by (3), hence their addition does not change the feasible region of the LP relaxation of $M1^+$ with respect to that of M1. We now apply Dantzig Wolfe Reformulation (DWR) to $M1^+$ (see [7]) through a convexification of

constraints (4) and (19). This is equivalent to impose the LP relaxation of such reformulation to be equal to the intersection of constraints (3) with the convex hull of constraints (4), (5), (6) and (19). The resulting model is exactly M3. One of the basic properties of DWR is that the LP relaxation of the reformulated model is as strong as the original model. Moreover, since the convexified region does not have the integrality property (i.e., the constraints (19) strictly contain their convex hull), this fact implies that there exists at least one objective function for which the reformulation of M1⁺ (i.e., M3) has a strictly better LP relaxation value than M1. A similar reasoning applies to models M2_A and M2_B, i.e., after adding constraints (19), M3 can be obtained via a convexification of constraints (9) and (19). $\square$

For the sake of completeness, we consider again the instance of Example 1. In this case, the optimal solution of LP3 is $\xi_{S_1}^* = 0.047$ $\xi_{S_2}^* = 1$ where the two subsets are $S_1 = \{2\}$ and $S_2 = \{3, 4\}$, and belong to classes 1 and 2, respectively. For these item sets we have $P_{S_1} = 74$, $P_{S_2} = 132$, $W_{S_1} = 85$ and $W_{S_2} = 148$. Thus, the optimal solution value is $U^3 = 135.482$, i.e., it is lower than U^1 , U_A^2 and U_B^2 , which are 144.254, 140.183 and 139.5, respectively (see Proposition 1).

3 Efficient computation of upper bounds for KPS

A natural way to compute an upper bound on the optimal solution value of a KPS instance is to solve the LP relaxation of the models introduced in Section 2 using a general LP solver. In this section we present effective combinatorial algorithms for solving the LP relaxation of the models above with no need of an external LP solver.

3.1 Solving the LP relaxation of M1

In this section we consider the LP relaxation of model M1. We observe that [22] introduces some properties of an optimal solution to LP1 and derives a combinatorial algorithm for its solution. However, no detailed description nor analysis on the computational complexity of the algorithm is given in [22]. Thus, for the sake of completeness, we report a pseudo-code of the algorithm in Figure 2. In addition, we show that the algorithm can be executed in linear time, thus improving the time complexity over the result claimed in [1]. Finally, we propose a strengthened relaxation that can be computed in constant time if an optimal solution to LP1 is known.

In this algorithm, one has to compute, for each class i , the *break item* b_i and the associated *macro-item* I_i . The former is the first item of the class for which the ratio between the cumulative profit and the cumulative weight is larger than the profit over weight ratio of all subsequent items (if no such item exists, the break item is the last item of the class), while the latter is an artificial item that simulates the profit and weight incurred when taking all items $j \in [\alpha_i, b_i]$ at the same value. For this reason, its profit (resp. weight) is equal to the sum of the profits (resp. weights) of all items $j \in [\alpha_i, b_i]$, plus the cost and capacity required for using the class. Given these figures, the algorithm adopts the well-known strategy for computing an optimal solution

Algorithm LP1:initialize: $\bar{N} := \emptyset$;**for each** class $i \in I$ **do** compute the break item b_i ; define the macro-item I_i ; $\bar{N} := \bar{N} \cup \{I_i\} \cup \{b_i + 1, \dots, \beta_i\}$;**end do**Solve the LP relaxation of the KP instance defined by item set $\bar{N}$, and let θ be the associated solution;**for each** class $i \in I$ **do** set $y_i^* = \theta_{I_i}$ and $x_j^* = \theta_{I_i} \quad \forall j = \alpha_i, \dots, b_i$; set $x_j^* = \theta_j \quad \forall j = b_i + 1, \dots, \beta_i$;**end do****Figure 2:** Algorithm to compute an optimal solution to LP1.

to the LP relaxation of KP, i.e., it takes one (either original or macro) item at a time until the entire capacity is used. The last selected item (or macro item) can then be taken at a fractional value. Finally, an optimal solution to LP1 is derived, observing that, for each class i , taking the associated macro-item at a given value, say θ_{I_i} corresponds to taking at the same value both y_i and all x_j variables associated with items $j \in [\alpha_i, b_i]$.

In an optimal LP1 solution, though many items may be taken at a fractional value, at most one y variable may be fractional. A similarity of LP1 with the LP relaxation of KP is given by the following observation:

Observation 1. An optimal solution to LP1 can be computed in $O(n)$ time.

To reach the claimed complexity, it is enough to observe that, for each class $i \in I$, the break item b_i can be computed in $O(n_i)$ time. This can be done with a split argument similar to that proposed by [3] for finding the critical item in a KP, yielding an overall $O(n)$ time complexity. As the resulting number of items to pack in the second step cannot be larger than n , this gives a linear time algorithm.

We conclude this section showing a strengthened relaxation that exploits the fact that the optimal LP1 solution has at most one fractional y variable and that, in any integer solution, this variable must take either value 0 or 1. Let $p(t)$ and $w(t)$ denote the profit and the weight, respectively, of the (either original or cumulative) item that is selected at each iteration t of the Dantzig's algorithm used to compute the LP1; in addition, let $\bar{t}$ be the number of iterations executed by the algorithm and $\bar{c}$ denote the residual capacity before inserting the last item. Similar to the MT bound proposed for KP by [16], we can derive the following upper bound for KPS

$$\overline{UB} = \max\{UB_0, UB_1\} \quad (20)$$

where

$$UB_0 = \sum_{t=1}^{\bar{t}-1} p(t) + \bar{c} \frac{p(\bar{t}+1)}{w(\bar{t}+1)} \quad \text{and} \quad UB_1 = \sum_{t=1}^{\bar{t}-1} p(t) + \left(p(\bar{t}) - (w(\bar{t}) - \bar{c}) \frac{p(\bar{t}-1)}{w(\bar{t}-1)} \right)$$

represent an upper bound on the optimal solution value when the fractional item is fixed to 0 and 1, respectively. In case $\bar{t} = 1$, this improved bound cannot be computed. Otherwise, it can be easily seen that $\overline{UB}$ dominates the bound produced by LP1 and that the computational effort for computing this bound is negligible if an optimal solution to LP1 has been computed.

3.2 Solving the LP relaxation of M2

Similar to Section 3.1, it is possible to compute an upper bound on the optimal solution of a KPS instance with a combinatorial algorithm based on the LP relaxation of model M2. In particular, we will denote by RLP2 the relaxation by LP2 removing the upper bound on variables y_i , i.e., replacing constraints (8) with $y_i \geq 0$ ($i \in I$).

Observation 2. There exists an optimal solution of RLP2, say (x^*, y^*) , such that

$$y_i^* = \frac{\sum_{j \in K_i} \bar{w}_j x_j^*}{\bar{C}_i^{\bar{w}}} \quad \forall i \in I. \quad (21)$$

It is easy to see that, for a given class i , constraint (9) imposes the above lower bound for variable y_i^* , and that increasing y_i^* with respect to this value produces a decrease of the solution value.

Based on Observation 2 one can reformulate RLP2 by substituting y variables; this yields to the following model

$$\max \left\{ \sum_{j \in N} \tilde{p}_j x_j : \sum_{j \in N} \tilde{w}_j x_j \leq C, x_j \in [0, 1], j \in N \right\},$$

where

$$\tilde{p}_j = p_j - \frac{f_{t_j}}{\bar{C}_{t_j}^{\bar{w}}} \bar{w}_j \quad \text{and} \quad \tilde{w}_j = w_j + \frac{s_{t_j}}{\bar{C}_{t_j}^{\bar{w}}} \bar{w}_j. \quad (22)$$

This model corresponds to the LP relaxation of a knapsack problem, which can be solved efficiently using again Dantzig's algorithm in linear time. Once an optimal solution, say x^* , is computed for the relaxation above, variables y^* can be computed a posteriori according to (21).

We conclude this section observing that, to the best of our knowledge, no combinatorial algorithms are available to solve LP2, whose solution requires instead the use of an LP solver. However, our computational experiments showed that the quality of the bound associated to LP2 is comparable with that obtained solving RLP2, for which our combinatorial algorithm is available. Finally, we mention that another relaxation of model M2 exists in which the integrality constraint is dropped for y variables only, while x variables are required to be binary;

by definition this relaxation dominates RLP2. Solving this relaxation requires the solution of a KP (NP-hard problem), possibly defined by non-integer profits and weights. Extensive computational tests show that this relaxation produces only marginal improvements, while the computational effort for solving the relaxation can be considerably larger than for RLP2.

3.3 Solving the LP relaxation of M3

Model M3 has exponentially many ξ_S variables ($i \in I, S \in \mathcal{S}_i$), which cannot be explicitly enumerated for large-size instances. *Column Generation* (CG) techniques are then necessary to efficiently solve its linear programming relaxation. In the following we discuss the CG framework for M3, and refer the interested reader to [7] for further details on CG.

Model (14)–(16) and (18), initialized with a subset of variables containing a feasible solution, is called *Restricted Master Problem* (RMP). Additional new variables, needed to solve LP3 to optimality, can be obtained by separating the following dual constraints:

$$W_S \lambda + \pi_i \geq P_S \quad i \in I, S \in \mathcal{S}_i, \quad (23)$$

where π_i ($i \in I$) is the dual variable associated with the i -th constraint (16) and λ is the dual variable associated with constraint (15). Accordingly, CG performs a number of iterations, until no violated dual constraint exist. At each iteration, the so-called *Pricing Problem* (PP) associated with each class $i \in I$ is solved. This problem asks to determine (if any) a subset $S^* \in \mathcal{S}_i$ for which the associated dual constraint (23), is violated, i.e., such that

$$\sum_{j \in S^*} (p_j - \lambda^* w_j) > \pi_i^* + \lambda^* s_i + f_i, \quad (24)$$

where π_i^* ($i \in I$) and λ^* are the dual variables values associated to the current solution of the RMP.

The pricing problem for class i asks for determining a subset of items $S^* \in \mathcal{S}_i$ that maximizes the left-hand-side of (24), and checking if this is larger than $\pi_i^* + \lambda^* s_i + f_i$. As such, finding the maximally violated dual constraint can be modeled as a KP, where each item $j \in K_i$ has profit $p_j - \lambda^* w_j$ and weight w_j . Using binary variables θ_j ($j \in K_i$), the problem reads as follows:

$$\tau^* = \max \left\{ \sum_{j \in K_i} (p_j - \lambda^* w_j) \theta_j : \sum_{j \in K_i} w_j \theta_j \leq C - s_i, \theta_j \in \{0, 1\}, \forall j \in K_i \right\}, \quad (25)$$

where $\theta_j = 1$ if and only if item j belongs to subset S^* . All variables with negative reduced costs that are generated, i.e., such that $\tau^* > \pi_i^* + \lambda^* s_i + f_i$ (if any), are added to the RMP, which is then re-optimized, according to a classic column generation scheme. If no column with negative reduced cost exists, the RMP is optimally solved and its solution (value) corresponds to the linear programming relaxation (value) of M3.

Since the pricing problems ask for the solution of a KP for each class, solving RMP (and, eventually, LP3) is weakly NP-hard. Computing an optimal solution for RMP at each CG

iteration corresponds to solving the LP relaxation of the classical ILP formulation for the MCKP (see also section 2.3) for which a linear-time algorithm is known in the literature (see [13]). We use this algorithm to compute an optimal RMP primal solution and to identify the so-called *critical item*. Since in LP3, variables are associated to subset of items instead of items, we will denote the critical item as the *critical subset*. Finally, an optimal RMP dual solution can be characterized thanks to the following proposition:

Proposition 3. *Let $\hat{S}$ be the critical subset in an optimal primal solution of RMP. Then, an optimal solution to the associated dual is the following:*

$$\lambda^* = \frac{P_{\hat{S}} - P_{\hat{S}-1}}{W_{\hat{S}} - W_{\hat{S}-1}} \quad \pi_i^* = \max_{S \in \mathcal{S}_i} \{P_S - W_S \lambda^*\}.$$

Proof. It is easy to verify that (λ^*, π^*) satisfies the dual constraints. We hence need to show that the primal and dual solutions have same objective value.

Let us denote by $\hat{i}$ the class associated with the critical subset $\hat{S}$. The value of the optimal primal solution can be written as follows:

$$\begin{aligned} \sum_{i \in I} \sum_{S \in \mathcal{S}_i} P_S \xi_S &= \sum_{i \in I, i \neq \hat{i}} \sum_{S \in \mathcal{S}_i} P_S \xi_S + P_{\hat{S}-1} \xi_{\hat{S}-1} + P_{\hat{S}} \xi_{\hat{S}} = \sum_{i \in I, i \neq \hat{i}} \sum_{S \in \mathcal{S}_i} P_S \xi_S + P_{\hat{S}-1} + (P_{\hat{S}} - P_{\hat{S}-1}) \xi_{\hat{S}} \\ &= \sum_{i \in I, i \neq \hat{i}} \sum_{S \in \mathcal{S}_i} P_S \xi_S + P_{\hat{S}-1} + \frac{C_r(P_{\hat{S}} - P_{\hat{S}-1})}{W_{\hat{S}} - W_{\hat{S}-1}} = \sum_{i \in I, i \neq \hat{i}} \sum_{S \in \mathcal{S}_i} P_S \xi_S + P_{\hat{S}-1} + C_r \lambda^* \end{aligned}$$

Since $C_r = C - \left(\sum_{i \in I, i \neq \hat{i}} \sum_{S \in \mathcal{S}_i} W_S \xi_S + W_{\hat{S}-1} \right)$ we have

$$\sum_{i \in I} \sum_{S \in \mathcal{S}_i} P_S \xi_S = \sum_{i \in I, i \neq \hat{i}} \sum_{S \in \mathcal{S}_i} (P_S - \lambda^* W_S) \xi_S + C \lambda^* + (P_{\hat{S}-1} - \lambda^* W_{\hat{S}-1}) \leq \sum_{i \in I} \pi_i^* + C \lambda^*$$

where the latter inequality derives from the definition of π_i^* variables and from the fact that, for each class i , exactly one subset is selected in the primal solution. As the objective function of the dual is $C \lambda + \sum_{i \in I} \pi_i$, the weak duality theorem ensures that (π_i^*, λ^*) is an optimal dual solution. $\square$

4 Exact solution of KPS

In this section we describe an exact approach to KPS based on branch-and-bound(-and price) techniques. Section 4.1 describes the way each branching node is evaluated, whereas Section 4.2 shows how a local upper bound is computed at each branching node.

4.1 Node exploration

Our enumerative algorithm is based on the observation that KPS reduces to KP in case the set of item classes to be selected is given. This suggests a branching rule in which first-stage

decisions are associated with the classes, whereas variables associated with items are treated as second-stage variables. For the sake of simplicity, in this section we will refer to the first formulation of KPS, i.e., we will make use of x and y variables to refer to selection of items and classes, respectively.

Figure 3 reports the pseudocode of the algorithm that is executed at each node of the tree. We first solve the LP relaxation at the current node and check whether the node can be fathomed, comparing the local upper bound with the incumbent solution, say z^* . In case enumeration must continue, we use a branching scheme similar to the one proposed by [11] for KP: at the root node we sort the classes according to non-increasing profit over weight ratio of the associated macro-item. At each node, we take the first y variable that is not fixed by branching and define two descendant nodes by fixing this variable to 1 and 0, respectively. Subsequent nodes, if any, are explored in the order they are generated, according to a depth-first strategy. Finally, if all the y variables are fixed by branching, a backtracking is executed.

Algorithm *Solve_node*:

```
// LP solution and possible fathoming
solve the LP relaxation at the current node;
let  $(x^*, y^*)$  denote an optimal LP solution, and  $U$  the associated value;
if  $U \leq z^*$  then fathom the node and return;
else
    // possible heuristic solution
    if all  $y^*$  variables are integer then
        solve a KP instance defined by items in the selected classes;
        let  $z(KP)$  be the associated profit (including setup costs);
        if  $z(KP) > z^*$  then
            update  $z^* := z(KP)$ ;
            if  $U \leq z^*$  then fathom the node and return;
        endif
    endif
    // possible branching
    if all  $y$  variables are fixed by branching then fathom the node and return;
    else
        let  $i$  be the first class that is not fixed by branching;
        define two subproblems branching on variable  $y_i$ ;
    endif
return
```

Figure 3: Exploration of a branch-and-bound node.

Note that in case all the y^* variables are integer, a heuristic step is executed to determine the optimal solution of the KP instance defined by all items in the selected classes. This is not strictly required for the correctness of the algorithm: indeed, an alternative strategy exists in which the resulting KP instance is solved only at the leaf nodes of the branch-and-bound tree. However, our computational experience showed a degradation in the performances of the resulting algorithm, which is able to update the incumbent solution very rarely. Though KP is an NP-hard problem, effective codes for its solution can be found in the literature; in our

implementation we used the routine `combo` proposed by [18], which is the state-of-the-art for KP problems. Obviously this step, that allows to avoid explicit branching on the x variables, is not required if all the x^* variables are integer as well; in this case, the incumbent is updated and a backtracking is performed.

Finally, we observe that our scheme may require to branch on some y variable also in case all of them take an integer value in the current LP solution. The following example shows that this (apparently, unnatural) branching is necessary to ensure the correctness of the approach. For a given $M \geq 4$, consider an instance defined by two item classes, both having a unitary setup cost and capacity. The first class includes two items with $p_1 = M$, $w_1 = 1$, $p_2 = M$, $w_2 = M$, whereas the second class includes one item only, with $p_3 = 2$ and $w_2 = 2$. The knapsack capacity is equal to 5. The LP relaxation of M1 has $y_1 = 1$, $x_1 = 1$ and $x_2 = 3/M$, while the second class is not used, i.e., $y_2 = x_3 = 0$. The associated upper bound is $(M - 1) + 3 = M + 2$. On the contrary the optimal integer solution is $y_1 = y_2 = 1$, with items $x_1 = x_3 = 1$ and $x_2 = 0$; the optimal integer value is M .

As already observed by [22], the algorithm by [1] does not allow branching on integer variables, and may thus fail in finding the optimal solution in situations similar to the one depicted above.

4.2 Local upper bounds

In this section we describe the way in which the LP relaxation of the models above are solved at each node of the enumeration tree.

Solution of LP1 As branching conditions involve y variables only, the algorithm described in Section 3.1 for solving LP1 has to be modified as follows. At the root node we store all the original and macro-items, sorted according to profit over weight ratio. At the current node, the local upper bound can be computed simply scanning the list of $n + m$ items: cumulative items can be used only for classes that are not fixed by branching. Original items can be used only for items that have been selected by branching (i.e., such that $y_i = 1$), while items that belong to a class that is forbidden by branching should not be used in the solution. It is easy to check that the computation of the local LP solution takes $O(n)$ time as at the root node.

Solution of LP2 Similar to LP1, solving RLP2 to optimality at each node requires small modification: items of classes that have been fixed to zero must not be selected, whereas for classes that have been selected, the fixed cost and capacity have to be taken into account, and items have to be evaluated according to original profits and weights. Finally, for items that belong to the remaining classes one has to use profit and weights $\tilde{p}_j$ and $\tilde{w}_j$, respectively, see (22). Observe that this bound can still be computed in $O(n)$ time at each node after the root. In particular, one can define a copy of each item j with profit $\tilde{p}_j$ and weight $\tilde{w}_j$, to be used for evaluating item j in case the associated class t_j has not been fixed by branching. This doubled set of items is sorted at the beginning of the algorithm. At each node of the tree, one can scan this double list and insert only copies of items in classes that have not been fixed and items in classes that have been selected.

Solution of LP3 The same branching scheme can be used with LP3 as well. Since the y variables are not explicitly considered, the branching decision for a specific class i can be imposed changing the right-hand-side of the associated constraint (16) in M3. To impose the condition $y_i = 1$, the constraint becomes:

$$\sum_{S \in \mathcal{S}_i} \xi_S = 1. \quad (26)$$

On the other side, to impose the condition $y_i = 0$, the constraint becomes:

$$\sum_{S \in \mathcal{S}_i} \xi_S = 0. \quad (27)$$

These modifications do not change the nature of the formulation nor the associated pricing problems PP. The effect of constraint (26) is just to remove the non-negativity constraint on the corresponding dual variable. From a practical viewpoint, imposing $y_i = 0$ corresponds to disregard item class i and all the associated items; this makes LP3 easier to solve, since a smaller number of pricing problems has to be solved at each iteration of the column generation process (see Section 3.3). Finally, since new variables may be generated within the branching nodes, the branch-and-bound algorithm becomes in this case a *branch-and-price* algorithm.

5 An Improved Dynamic Programming Algorithm

In this section we describe a way of improving the storage requirements and the computational performances of the Dynamic Programming algorithms proposed in [5]. The basic algorithm given in [5] consists of n stages, each having 2 states associated with each possible capacity value from 0 to C . Let j be an item of class i , i.e., $j \in K_i$. For each capacity value r , state $A(j, r)$ reports the optimal solution value of the sub-instance defined by item set $\{1, \dots, j\}$ and capacity r when class i is used, while state $B(j, r)$ gives the same figure when class i is not used (and the associated items cannot be selected). This scheme requires to store two matrices of size $n \times (C + 1)$, which makes the algorithm not suitable for instances with a large value of capacity and/or large numbers of items. To reduce the storage requirements, a second scheme was proposed in [5], that converts a KPS solution into an integer index. However, this can produce some slow-down in the performances of the approach. We refer the interest reader to [5] for further details on these two algorithms.

We now give some details about our implementation of the algorithm. The new algorithm will be computationally tested in Section 7.

To reduce memory requirements, we implemented an algorithm that does not require the *explicit* storage of the two matrices, but uses two vectors of size $C + 1$ each. This is possible as, at each stage j , only entries from column $j - 1$ are used to update the current stages. This space reduction may produce considerable improvements in terms of computing times too in practice, in particular for large-size problems. Our algorithm does not explicitly store the solution vector, but gives in output only the optimal solution value. This is coherent with what is done in the

state-of-the-art DP for KPS, recently proposed in [21]; in particular, this is the DP algorithm used as benchmark in our computational experiments (see Section 7). As observed in [21], an optimal solution vector can be retrieved after the dynamic programming algorithm is completed, simply adapting the general recursive storage reduction principle from [20], thus preserving both the original running time and space complexity. The reader is referred to [13, Sec. 3.3] for a detailed description of this scheme.

6 A relevant special case

In this section we introduce a special relevant case that may be encountered when solving KPS. This happens if, for each class $i \in I$, the following condition is satisfied

$$s_i + \sum_{j \in K_i} w_j \leq C \quad (28)$$

This means that, for each class i , all items of the class can be allocated into the knapsack.

We observe that KPS remains NP-hard also in case assumption (28) is valid. Moreover, this special case is relevant from a theoretical viewpoint: while KPS does not admit a polynomial time approximation algorithm with a bounded approximation ratio, there exists an FPTAS that can be derived when assumption (28) is satisfied (see, [21]). Finally, this situation is always satisfied for the instances in our testbed that are taken from the literature (see Section 7.1).

Therefore, for the rest of this section we will assume that (28) is valid.

Proposition 1. *Under assumption (28), U^1 dominates both U_A^2 and U_B^2 .*

Proof. Note that, for each class $i \in I$, all items in K_i can be inserted in the knapsack. Thus, we have $C_i^{\bar{w}} = \sum_{j \in K_i} \bar{w}_j$, i.e., the surrogate capacity can be computed in linear time. In order to show the result, we have to prove that every feasible solution to LP1 is feasible for LP2. This can be trivially proved as, for each class i , the surrogate constraint (9) in M2 can be obtained summing up constraints (4) associated with items $j \in K_i$ using non-negative coefficients $\bar{w}_j$. To conclude the proof, we observe that, for the second instance in the proof of Proposition 1, upper bound U^1 is strictly smaller than both U_A^2 and U_B^2 . $\square$

Proposition 2. *Under assumption (28), LP3 can be computed in $O(n)$ time.*

Proof. Consider a given class i . If all items in K_i can be inserted in the knapsack, the pricing problem (25) for a given λ^* has the following optimal solution:

$$\theta_j = \begin{cases} 1 & \text{if } p_j - \lambda^* w_j > 0 \\ 0 & \text{otherwise} \end{cases} \quad (j \in K_i)$$

Since items are sorted according to non-increasing profit over weight ratio, this means that all items $j \in [\alpha_i, \gamma_i(\lambda^*)]$ will be selected, where $\gamma_i(\lambda^*) = \min\{j \in K_i : p_j/w_j \leq \lambda^*\}$. Thus, at most n_i variables associated with class i have to be considered into the model—namely, for each item

$j \in K_i$, one variable corresponding to item set $[\alpha_i, j]$. Overall, model M3 is thus an MCKP with n variables, whose LP relaxation can be solved in $O(n)$ time using the algorithm presented by [8] and [23]. $\square$

Corollary 3. *Under assumption (28), $U^1 = U^3$.*

Proof. This follows from Proposition 2 and by observing that, in case (28) is satisfied, constraints (19) are equivalent to their convex hull; thus, the convexified region has the integrality property. This implies that no bound improvement can be obtained by the convexification of constraints (4) and (19). $\square$

7 Computational Experiments

In this section we report an extensive computational analysis on the performances of our algorithms. All codes were implemented using C and were run on an Intel Xeon E3-1220 V2 running at 3.10 GHz. We first describe the instances taken from the literature (see Section 7.1) and some new randomly generated instances (see Section 7.2) and then present the outcome of our experiments.

7.1 Instances from the literature

To the best of our knowledge, some sets of test instances have been proposed in the literature for knapsack problems with setup. The 180 randomly generated instances proposed by [19] are not publicly available and refer to KPS with additional upper bounds on the maximum weight that can be used for each item class. We generated this testbed of instances following the description of [19]. Thanks to the impressive improvements of commercial ILP solvers in the last decade, all these instances are now easily solved directly using a general-purpose ILP solver on model M1.

Much harder KPS instances have been proposed in [5]. These problems, publicly available at <https://sites.google.com/site/chebilkh/knapsack-problem-with-setup>, were generated to simulate realistic instances from an industrial application. In particular, these instances have been randomly generated with a number of items $n \in \{500, 1000, 2500, 5000, 10000\}$ and a number of classes $m \in \{5, 10, 20, 30\}$; ten instances have been generated for each pair (n, m) , thus producing a testbed of 200 problems. Item profits and weights have been generated so as to have strongly correlated instances, and the setup cost (resp. capacity) of each class is a random number correlated to sum of the profits (resp. weights) of the items in the class. Observe that this benchmark has also been used by other recent works on KPS (e.g., [6] and [21]), and that all instances in this set satisfy condition (28).

Recently, four additional classes of instances have been introduced in [6]. The authors kindly provided us all these instances in a private communication. These instances, that were generated according to the scheme given in [22], have been denoted as Classes 1, 2, 3 and 5, whereas Class 4 is used to refer to the problems proposed by [5]. Classes 1, 2 and 3 refer to uncorrelated (w_j

and p_j in $[10, 10000]$), correlated (w_j in $[10, 10000]$ and p_j in $[w_j - 1000, w_j + 1000]$) and strongly correlated (w_j in $[10, 100]$ and $p_j = w_j + 10$) instances, respectively. Each class contains 160 problems. For these instances, the number of item classes is $m \in \{50, 100\}$, while the number of items n_i for each class $i \in I$ is uniformly distributed in the ranges $[40, 60]$ and $[90, 110]$. The setup cost f_i and setup capacity s_i are defined accordingly to the following formulas: $f_i = e_1 \left(\sum_{j \in K_i} p_j \right)$ and $d_i = e_2 \left(\sum_{j \in K_i} w_j \right)$, where e_1 and e_2 are uniformly distributed in the intervals $[0.05, 0.15]$, $[0.15, 0.25]$, $[0.25, 0.35]$ and $[0.35, 0.45]$. Finally, Class 5 includes 100 larger strongly correlated instances with up to 100000 items and 200 classes, where setup costs and capacities are obtained taking $e_1 = e_2$ uniformly ranging in the interval $[0.15, 0.25]$. For all classes proposed in [6], the capacity C is an integer value randomly generated in the range $[0.4(\sum_{j \in N} w_j), 0.6(\sum_{j \in N} w_j)]$. It turns out that all the instances in the new benchmark satisfy property (28).

7.2 New instances

To enlarge our benchmark we generated two additional classes of instances.

The first class (denoted as Class 6) is obtained from computationally challenging KP instances, in which the item weights and profits (w_j and p_j , $j \in N$) are generated using the generator proposed in [18] (available at <http://www.diku.dk/~pisinger/codes.html>). More in details, we generated instances belonging to the following eight classes of KP instances:

1. *Uncorrelated*: w_j u.r. in $[1, \bar{R}]$, p_j u.r. in $[1, \bar{R}]$.
2. *Weakly correlated*: w_j u.r. in $[1, \bar{R}]$, p_j u.r. in $[\max\{1, w_j - \bar{R}/10\}, w_j + \bar{R}/10]$.
3. *Strongly correlated*: w_j u.r. in $[1, \bar{R}]$, $p_j = w_j + \bar{R}/10$.
4. *Inverse strongly correlated*: p_j u.r. in $[1, \bar{R}]$, $w_j = p_j + \bar{R}/10$.
5. *Almost strongly correlated*: w_j u.r. in $[1, \bar{R}]$, p_j u.r. in $[w_j + \bar{R}/10 - \bar{R}/500, w_j + \bar{R}/10 + \bar{R}/500]$.
6. *Subset-sum*: w_j u.r. in $[1, \bar{R}]$, $p_j = w_j$.
7. *Even-odd subset-sum*: w_j even value u.r. in $[1, \bar{R}]$, $p_j = w_j$, C odd.
8. *Even-odd strongly correlated*: w_j even value u.r. in $[1, \bar{R}]$, $p_j = w_j + \bar{R}/10$, C odd,

where $\bar{R} = 1000$ and u.r. stands for “uniformly random integer”.

In order to transform a KP instance into a KPS instance, we defined the knapsack capacity with a formula similar to the one used in [6], i.e., $C = e_3 \sum_{j \in N} w_j$ where $e_3 \in \{0.45, 0.5, 0.55\}$. The setup cost f_i and setup capacity s_i are also defined similarly to [6], setting $e_1 = e_2 = 0.05$. The items are uniformly partitioned among the classes, i.e., $|K_i| = \frac{n}{m}$ ($i \in I$). In this way, for each size $n \in \{5000, 10000, 20000, 100000, 200000\}$ and $m \in \{5, 10\}$, we generated 24 instances. In total this new testbed is composed of 240 instances. All these instances as well satisfy property (28) discussed in Section 6.

The second class (denoted as Class 7) has been introduced to consider instances for which property (28) is violated. To the best of our knowledge, this is the first benchmark of instances with this characteristic. These instances are derived from the instances of Class 5, that are by far the hardest KPS problems from the existing literature. In particular, for each instance of Class 5, we define a twin instance in Class 7 as follows. We keep the same knapsack capacity and the same item set as in the original instance. Then, we include in the first class all the items of the first σ_1 classes in the original instance, where $\sigma_1 = \min\{k : \sum_{i=1}^k (s_i + \sum_{j \in K_i} w_j) > c\}$. The setup cost (resp. capacity) for the new class is the sum of the σ_1 setup costs (resp. capacities) of the original classes. In the same way, we define the second class by considering original classes $\sigma_1 + 1, \dots, \sigma_2$, where $\sigma_2 = \min\{k : \sum_{i=\sigma_1+1}^k (s_i + \sum_{j \in K_i} w_j) > c\}$. In order to avoid instances with a too small number of classes (that could bias the comparison, as explained later), all the remaining classes $i = \sigma_2 + 1, \dots, m$ are left unchanged.

All the instances considered in our computational analysis are available at the following address: <http://or.dei.unibo.it/library/knapsack-problem-setup>.

7.3 Solving the LP relaxation of the models

Our first set of experiments is aimed at evaluating the quality of the upper bound resulting from the LP relaxation of the ILP models given in Section 2. To evaluate the effect of condition (28), we consider both the instances proposed in [5] and the newly generated instances in Class 7.

Table 1 refers to instances of Class 4 and reports, for each model, the associated percentage gap, computed as $\% \text{gap} = 100 * \frac{U - z^*}{z^*}$, where U and z^* denote the value of the relaxation and of the optimal solution, respectively. In this table we do not distinguish between LP1 and LP3, that produce the same upper bound (see, Corollary 3). Columns RLP2_A and RLP2_B correspond to the relaxations of models M2_A and M2_B, respectively (see Section 3.2). All figures report average values over the 50 instances having the same values for m .

Instances	RLP2_A	RLP2_B	LP1-LP3
m	% gap	% gap	% gap
5	11.130	7.432	0.944
10	11.457	7.442	1.198
20	11.175	6.977	0.393
30	10.823	5.809	0.189

Table 1: LP relaxation of the models on instances of Class 4.

These results confirm the theoretical dominance among the relaxations: LP1 and LP3 provide a very tight upper bound on the optimal value, while, on average, RLP2 (in both variants) yields a poor approximation. Finally, we note that while the combinatorial algorithms we used are extremely fast both from a theoretical and from a practical viewpoint, whereas solving these relaxations with a state-of-the-art commercial solver may require a non negligible computing time, that may be relevant in case millions of branching nodes have to be explored.

Table 2 refers to instances of Class 7, for which property (28) is violated. For this reason, we report the average gaps of both LP1 and LP3 separately, and give the same information as in Table 1. Instances are grouped according to the number of items, i.e., each line reports average values over 70 instances with the same value of n .

Instances	LP1	RLP2_A	RLP2_B	LP3
n	% gap	% gap	% gap	% gap
20000	2.605	8.168	10.583	0.650
50000	2.317	8.179	10.454	0.664
100000	1.800	7.975	9.432	0.596

Table 2: LP relaxation of the models on instances of Class 7.

The results in Table 2 show that, for instances that do not satisfy condition (28), model M3 can produce a relaxation with an upper bound that is considerably tighter than the one provided by the relaxation of M1. Note however that the computation of this relaxation may be computationally more expensive to obtain than U^1 , and this may lead to less efficient performances when large branch-and-bound trees are explored.

7.4 Combinatorial algorithms

In this section we evaluate the computational performances of exact approaches for KPS. In a set of preliminary computational experiments, we used IBM-ILOG Cplex 12.6 on models M1 and M2 and obtained results that are coherent with those reported in [5, 6]: the direct application of an ILP solver is typically unable to solve instances with more than 2500 items in a systematic way, even using state-of-the-art codes. For this reason, we do not report detailed results for this approach in what follows. As to the combinatorial algorithms, we consider $B\&B_{LP1}$ and $B\&P_{LP3}$, that denote the Branch-and-Bound and Branch-and-Price algorithms based on the LP relaxations of models M1 (strengthened as shown in Section 3.1) and M3, respectively, and the improved dynamic programming algorithm (DP in the following) described in Section 5. Preliminary computational experiments showed that the branch-and-bound algorithm based on model M2 is dominated by the other approaches; hence, we do not report results for this algorithm.

For this analysis, we used the instances of Class 4, and compared our algorithms with the exact algorithms given in [6] and in [21]. In the following, these algorithms will be denoted as DSS and PS, respectively. Both algorithms showed to outperform the dynamic programming in [5] on our benchmark; for this reason, the latter is excluded from comparison. In a personal communication, the authors of [6] provided us with the implementation of their algorithm, which allows a better performance evaluation with our new exact algorithms. Since all our algorithms are sequential, in a first round of tests we ran DSS on our machine using 1 thread only and denote by DSS_{1TH} the resulting code (see the next section for a multi-thread performance analysis). We observe that the results for algorithm PS are taken from [21] and were obtained on an Intel i5 CPU running at 3.2 GHz with 16 GB of RAM.

Instances		B&B _{LP1}		B&P _{LP3}		DP		DSS _{1TH}		PS	
m	n	Time (sec.)		Time (sec.)		Time (sec.)		Time (sec.)		Time (sec.)	
		avg	max	avg	max	avg	max	avg	max	avg	max
5	500	0.00	0.00	0.00	0.01	0.02	0.02	0.08	0.10	0.02	0.02
	1000	0.00	0.00	0.00	0.01	0.05	0.06	0.13	0.17	0.07	0.08
	2500	0.00	0.01	0.01	0.01	0.25	0.25	0.34	0.35	0.54	0.56
	5000	0.00	0.01	0.01	0.01	0.90	0.91	0.93	1.07	2.00	2.16
	10000	0.01	0.01	0.01	0.02	3.66	3.69	3.50	4.17	8.67	8.98
10	500	0.00	0.01	0.00	0.01	0.02	0.02	0.14	0.24	0.01	0.02
	1000	0.01	0.01	0.01	0.01	0.06	0.06	0.15	0.21	0.07	0.08
	2500	0.01	0.02	0.01	0.02	0.25	0.26	0.35	0.44	0.49	0.50
	5000	0.01	0.02	0.01	0.02	0.93	0.94	0.86	0.92	1.79	1.84
	10000	0.02	0.03	0.02	0.03	3.70	3.76	2.76	3.02	7.15	7.33
20	500	0.00	0.00	0.00	0.00	0.02	0.02	0.12	0.18	0.02	0.02
	1000	0.20	0.52	0.15	0.37	0.06	0.06	0.27	0.44	0.07	0.07
	2500	3.19	5.98	1.42	2.74	0.25	0.26	0.46	0.78	0.42	0.45
	5000	3.05	8.46	1.23	3.24	0.94	0.95	0.87	1.17	1.68	1.70
	10000	7.07	14.63	2.61	5.18	3.76	3.81	2.64	2.88	6.69	6.71
30	500	0.00	0.02	0.01	0.03	0.02	0.02	0.36	1.20	0.02	0.03
	1000	3.02	6.45	2.71	5.60	0.06	0.06	0.88	2.77	0.06	0.07
	2500	18.95	70.47	10.00	37.77	0.25	0.26	0.52	0.67	0.45	0.46
	5000	1.38	7.86	0.66	3.65	0.94	0.95	1.02	1.59	1.66	1.68
	10000	11.90	78.98	4.87	30.83	3.76	3.77	2.92	4.44	6.58	6.74

Table 3: Sequential algorithms for the exact solution of KPS for instances of Class 4.

Table 3 reports the performance comparison of the sequential algorithms. Each line of the table refers to 10 instances with the same number of classes and items. For each algorithm we report the average and maximum computing time (in seconds) for solving the associated instances.

Results in Table 3 show that our enumerative algorithms are competitive with both DSS_{1TH} and PS on this benchmark. There are some specific sets of problems in which they are considerably faster than the previous approaches from the literature, in particular when the number of item classes is “small”, regardless of the number of items. We also observe that M3 is typically faster than M1. Indeed, as condition (28) is always verified on these instances, these two algorithms explore a similar number of nodes. However, the computation of LP3 is usually faster than that of LP1 due to two main factors: first, M3 is usually able to terminate the enumeration generating only few variables, hence the model maintains a small size. In addition, the *practical* complexity of computing LP3 is usually easier than the *theoretical* one, especially when some y variables have been fixed to zero by branching. In this case, M3 simply disregards the associated classes, thus saving computing time, while the algorithm for computing LP1 has to scan in any case the associated items, though inserting them in the solution is forbidden. Our improved dynamic programming DP has good performances when the number of items is small, regardless the number of classes. Given its $O(nC)$ complexity, the required computing time is strongly dependent on the number of items. A similar behaviour can be observed evaluating the results of the dynamic programming algorithm PS proposed in [21]. Finally, we note that both M1 and M3 may have large variability for what concerns the solution time, which is not the case for our dynamic programming algorithm DP.

7.5 Parallel algorithms

In this section we address the more challenging instances recently proposed in [6] as well as the new instances described in Section 7.2. As algorithm DSS is based on an ILP solver and may take advantage of the availability of a multi-thread architecture, we ran this code on our machine allowing for 4 cores (as in [6]), and denote by $\text{DSS}_{4\text{TH}}$ the resulting code. In our tests we used IBM-ILOG Cplex 12.6, that is a more recent release than the 12.5 one used in [6]. For this reason, computing times reported in Tables 4–6 for algorithm $\text{DSS}_{4\text{TH}}$ may be different (typically, smaller) than those reported in [6].

To have a fair comparison, we implemented a parallel variant of our algorithm (FMT in the following) that executes algorithms $\text{B\&B}_{\text{LP1}}$, $\text{B\&P}_{\text{LP3}}$ and DP in parallel, and halts execution as soon as one of the 3 terminates. In particular, in our implementation we used OpenMP, an API that allows to add parallelism into existing source codes in a simple and flexible way. Furthermore, we report results for a parallel variant of algorithm DSS, in which this code is run in single-thread mode and one additional thread is used for running algorithm DP. We denote by $\text{DSS}_{1\text{TH}}+\text{DP}$ the resulting algorithm, which has been implemented using OpenMP as for FMT. The rationale of this algorithm is to allow a comparison with another parallel algorithm that exploits different strategies.

Table 4 reports the results on instances of Class 4, i.e., the classical KPS benchmark from the literature, proposed in [5] and already considered in Table 3. The table reports the same information as in Table 3, with an additional column (**best**) for algorithm FMT, in which a 3-field entry reports the number of instances for which $\text{B\&B}_{\text{LP1}}$, $\text{B\&P}_{\text{LP3}}$ and DP is the best algorithm.

On these instances, algorithm FMT is typically faster than $\text{DSS}_{4\text{TH}}$, with a considerable speedup for instances with $m \leq 10$. Observe that, in some cases, algorithm $\text{DSS}_{4\text{TH}}$ takes fully advantage of the parallelization and reaches a speedup that is even larger than the number of threads that are used. Indeed, modern ILP solvers adopt different strategies according to the number of available threads. For example, IBM-ILOG Cplex 12.6 includes cut generation procedures that are executed at the root node when multiple threads are available, and that are aimed at reducing the variability of the solver (see [9] for details). It is worth mentioning that FMT takes full advantage of its parallelism: for all instances with $m \leq 10$ the best algorithm is either $\text{B\&B}_{\text{LP1}}$ or $\text{B\&P}_{\text{LP3}}$, whereas for large values of m (in particular, for $n = 1000$ and $n = 2500$) DP is effective in reducing the computing time. Finally, we note that a similar behaviour does not occur for algorithm $\text{DSS}_{1\text{TH}}+\text{DP}$: comparing its results with those of algorithm $\text{DSS}_{1\text{TH}}$ of Table 3, it can be noticed that DP yields a reduction of the computing times only for instances with small values of n , whereas for the hardest instances a 2x slowdown is observed. This computational behaviour can be explained by the fact that, for large instances, both IBM-ILOG Cplex 12.6 and DP may require a large amount of memory, which deteriorates the performances. Since a similar behaviour is experienced for all the instances in our benchmark, we refrain from reporting any further result for algorithm $\text{DSS}_{1\text{TH}}+\text{DP}$ hereafter.

Tables 5 and 6 report the results for all the classes of instances proposed in [6]. Each line of

Instances		FMT			DSS _{4TH}		DSS _{1TH+DP}	
m	n	Time (sec.)		best	Time (sec.)		Time (sec.)	
		avg	max		avg	max	avg	max
5	500	0.00	0.00	10/0/0	0.09	0.12	0.04	0.04
	1000	0.00	0.00	10/0/0	0.15	0.21	0.12	0.12
	2500	0.00	0.01	10/0/0	0.35	0.40	0.53	0.54
	5000	0.00	0.01	10/0/0	0.48	0.50	1.82	1.88
	10000	0.01	0.01	10/0/0	0.85	0.98	6.94	7.43
10	500	0.00	0.01	10/0/0	0.14	0.20	0.04	0.04
	1000	0.01	0.01	0/10/0	0.16	0.20	0.12	0.13
	2500	0.01	0.02	0/10/0	0.35	0.42	0.54	0.54
	5000	0.01	0.02	5/5/0	0.54	0.58	1.73	1.86
	10000	0.02	0.03	3/7/0	0.90	0.96	5.59	6.09
20	500	0.00	0.00	10/0/0	0.14	0.21	0.04	0.04
	1000	0.06	0.06	0/2/8	0.26	0.43	0.12	0.13
	2500	0.23	0.25	0/1/9	0.44	0.74	0.54	0.56
	5000	0.68	0.98	0/5/5	0.65	0.74	1.71	1.96
	10000	2.41	3.88	0/7/3	1.01	1.12	5.35	5.80
30	500	0.00	0.02	9/0/1	0.40	1.29	0.04	0.04
	1000	0.06	0.06	0/0/10	0.53	1.37	0.12	0.13
	2500	0.23	0.26	0/1/9	0.49	0.59	0.54	0.55
	5000	0.36	0.98	0/8/2	0.79	1.05	1.83	1.96
	10000	1.65	3.89	0/7/3	1.59	2.32	5.79	7.61

Table 4: Parallel algorithms on instances of Class 4.

these tables gives the average and the maximum computing time over ten instances with similar characteristics.

Instances			Class 1				Class 2				Class 3			
			FMT		DSS _{4TH}				FMT		DSS _{4TH}			
m	n_i	setup	Time (sec.)		Time (sec.)		Time (sec.)		Time (sec.)		Time (sec.)		Time (sec.)	
			avg	max	avg	max	avg	max	avg	max	avg	max	avg	max
50	[40-60]	[0.05-0.15]	0.00	0.00	0.24	0.33	0.01	0.03	0.31	0.49	0.18	0.32	0.79	1.96
		[0.15-0.25]	0.00	0.00	0.29	0.39	0.01	0.04	0.32	0.47	0.17	0.32	0.70	1.30
		[0.25-0.35]	0.00	0.00	0.23	0.28	0.02	0.07	0.36	0.52	0.16	0.30	0.49	0.99
		[0.35-0.45]	0.00	0.00	0.26	0.41	0.01	0.01	0.34	0.53	0.11	0.30	0.94	2.76
50	[90-110]	[0.05-0.15]	0.00	0.00	0.63	0.75	0.07	0.24	0.87	1.09	0.52	1.28	1.23	3.29
		[0.15-0.25]	0.00	0.00	0.65	0.81	0.48	4.16	0.79	0.96	0.52	1.23	1.15	2.56
		[0.25-0.35]	0.00	0.01	0.56	0.81	2.31	11.71	0.90	1.32	0.43	1.14	0.79	1.05
		[0.35-0.45]	0.00	0.00	0.44	0.62	0.01	0.05	0.68	0.93	0.91	1.26	1.10	1.81
100	[40-60]	[0.05-0.15]	0.00	0.00	0.60	1.20	0.05	0.16	0.64	0.84	0.55	1.24	2.10	7.65
		[0.15-0.25]	0.00	0.01	0.54	0.83	0.02	0.05	0.70	0.90	0.59	1.04	1.23	3.16
		[0.25-0.35]	0.00	0.01	0.50	0.67	0.10	0.42	0.78	1.09	0.83	1.21	1.44	3.50
		[0.35-0.45]	0.00	0.00	0.43	0.58	0.01	0.01	0.54	0.71	0.61	1.22	4.39	26.66
100	[90-110]	[0.05-0.15]	0.01	0.01	1.30	1.52	1.78	6.07	1.47	1.85	2.76	4.89	2.54	4.42
		[0.15-0.25]	0.01	0.01	1.15	1.35	4.50	40.65	1.67	3.10	2.92	4.61	2.64	7.24
		[0.25-0.35]	0.01	0.02	1.11	1.69	4.86	23.77	1.56	2.49	2.14	4.43	2.25	4.58
		[0.35-0.45]	0.00	0.01	0.89	1.16	0.21	1.58	1.47	2.08	2.75	4.26	2.72	5.66

Table 5: Parallel algorithms on instances of Classes 1, 2 and 3.

In Table 5 we compare the performances of FMT and DSS_{4TH} for the first three classes, that

include 480 instances obtained using the same instance generator parameters for m , n_i and setup. While $m \in \{50, 100\}$ is the number of classes, n_i represents the interval used to determine the number of items for each class $i \in I$ and column “setup” gives the interval used for parameters e_1 and e_2 , see Section 7.1. These results show that our algorithm FMT is competitive with algorithm $\text{DSS}_{4\text{TH}}$: for instances of Class 1, FMT outperforms $\text{DSS}_{4\text{TH}}$, often by orders of magnitude. Overall these 480 instances, the average computing times of the algorithms are 0.63 and 1.03 respectively, and FMT is faster than $\text{DSS}_{4\text{TH}}$ in 431 cases.

Instances		FMT			$\text{DSS}_{4\text{TH}}$	
m	n	Time (sec.)		best	Time (sec.)	
		avg	max		avg	max
5	20000	0.01	0.01	10/0/0	1.60	1.95
	50000	0.02	0.04	10/0/0	6.42	9.33
	100000	0.04	0.11	10/0/0	42.92	48.08
10	20000	0.03	0.05	1/9/0	1.40	1.65
	50000	0.07	0.09	2/8/0	4.08	5.94
	100000	0.12	0.16	2/8/0	12.90	23.84
20	20000	4.29	10.33	0/10/0	1.79	2.05
	50000	9.43	22.85	0/10/0	5.21	5.91
	100000	17.58	44.03	0/10/0	15.62	23.22
30	20000	4.10	16.91	0/9/1	2.14	2.70
	50000	23.10	106.92	0/9/1	4.97	6.15
	100000	120.25	429.79	0/8/2	15.36	23.38
50	20000	7.46	17.16	0/6/4	2.05	3.09
	50000	53.22	107.70	0/6/4	6.40	7.91
	100000	85.02	432.17	0/9/1	15.91	19.65
100	20000	9.32	17.16	0/5/5	4.46	10.30
	50000	47.82	108.02	0/7/3	8.41	26.89
	100000	161.53	432.80	0/7/3	11.07	15.88
200	20000	12.50	17.33	0/5/5	6.32	16.42
	50000	58.44	108.75	0/6/4	14.62	33.58
	100000	224.17	435.15	0/5/5	15.54	27.29

Table 6: Parallel algorithms on instances of Class 5.

Results on instances of Class 5 are given in Table 6. On this benchmark, algorithm $\text{DSS}_{4\text{TH}}$ performs slightly better than FMT on average. However, while the computing time of FMT seems to be highly dependent on the number of classes, the performances of $\text{DSS}_{4\text{TH}}$ are highly influenced by the number of items. For this reason, our approach turns out to be much faster than $\text{DSS}_{4\text{TH}}$ for all problems that have $m \leq 10$, independently on the number of items. This is not surprising, as our approach was originally designed to solve the instances proposed in [5], that have a small number of classes, and exploits the branching on the y variables. On the other hand, algorithm $\text{DSS}_{4\text{TH}}$ is based on the application of a general purpose ILP solver, which may get into troubles when the number of decision variables gets too large.

We now present the results for the new instances introduced in Section 7.2. Since these new instances are intended to represent challenging benchmarks, for all these experiments we set a time limit of 1200 CPU seconds per instance.

Each row in Table 7 shows the results for the 24 instances of Class 6 that have the same number of classes m and number of items n . For both FMT and $\text{DSS}_{4\text{TH}}$ we report the number of instances solved to proven optimality within the time limit, and the average and maximum computing times. Average computing times refer to instances solved to proven optimality only. T.L. means that at least one instance in the associated subset hits the given time limit.

Instances		FMT				$\text{DSS}_{4\text{TH}}$		
m	n	#opt	Time (sec.)		best	#opt	Time (sec.)	
			avg	max			avg	max
5	5000	24	0.00	0.00	24/0/0	22	0.70	T.L.
	10000	24	0.00	0.01	24/0/0	24	1.28	3.56
	20000	24	0.00	0.01	24/0/0	22	2.45	T.L.
	100000	24	0.02	0.05	21/3/0	24	19.05	60.18
	200000	24	0.03	0.07	19/5/0	24	73.90	287.79
10	5000	24	0.00	0.01	12/12/0	24	0.72	1.09
	10000	24	0.00	0.02	12/12/0	23	1.63	T.L.
	20000	24	0.01	0.06	11/13/0	23	2.55	T.L.
	100000	24	0.04	0.17	9/15/0	24	21.17	44.48
	200000	24	0.11	0.32	9/15/0	24	67.57	137.21

Table 7: Parallel algorithms on instances of Class 6.

These results confirm that for hard instances with a small number of classes algorithm FMT outperforms $\text{DSS}_{4\text{TH}}$. Indeed, the former is able to solve each of the 240 instances in Class 6 in at most half a second, whereas algorithm $\text{DSS}_{4\text{TH}}$ fails in solving 6 problems within the 1200-seconds time limit. As to the average computing times, FMT is always faster than $\text{DSS}_{4\text{TH}}$, often by some orders of magnitude. Indeed, though we compute average values only considering instances solved to proven optimality, FMT has an average computing time of 0.02 seconds, while $\text{DSS}_{4\text{TH}}$ requires about 50 seconds on average. Finally, we observe that, for these instances, DP is never the best algorithm; this may be expected as problems in Class 6 have a large number of items but a relatively small number of classes, and thus can be solved efficiently using combinatorial algorithms $\text{B\&B}_{\text{LP1}}$ and $\text{B\&P}_{\text{LP3}}$.

Table 8 reports the results of our experiments on the instances of Class 7. Since these problems are derived from those of Class 5, the instances are grouped as in Table 6, i.e., according to the number of items n and to the number of classes m in the original instances. For each group of 10 instances we also give the average number m^* of classes in the new instances; this number is in the range from 2 to 35.

The results in Table 8 show that FMT is typically faster than $\text{DSS}_{4\text{TH}}$ also for instances for which condition (28) does not hold. These instances have the same computational difficulty as those in Class 5 for algorithm FMT, and in this case too our algorithm largely outperforms $\text{DSS}_{4\text{TH}}$ (in some cases, up to 4 orders of magnitude) for problems with a small number of classes. **Finally observe that, contrary to the instances of Class 5, algorithm $\text{B\&P}_{\text{LP3}}$ is never the best algorithm among those used within FMT.**

Instances			FMT			DSS _{4TH}	
m	n	m^*	Time (sec.)		best	Time (sec.)	
			avg	max		avg	max
5	20000	2.0	0.00	0.01	10/0/0	3.25	6.46
	50000	2.0	0.01	0.03	10/0/0	21.34	45.40
	100000	2.0	0.01	0.02	10/0/0	123.15	229.97
10	20000	2.0	0.00	0.01	10/0/0	2.74	4.41
	50000	2.2	0.01	0.02	10/0/0	10.14	21.92
	100000	2.3	0.02	0.04	10/0/0	72.11	142.88
20	20000	4.0	0.00	0.01	10/0/0	2.60	3.05
	50000	4.0	0.01	0.03	10/0/0	13.43	16.50
	100000	4.2	0.02	0.05	10/0/0	41.06	56.30
30	20000	5.8	0.01	0.02	10/0/0	3.07	3.56
	50000	5.8	0.01	0.03	10/0/0	14.14	17.01
	100000	6.0	0.04	0.11	10/0/0	49.15	59.59
50	20000	9.0	0.02	0.03	10/0/0	3.88	4.41
	50000	9.6	0.04	0.11	10/0/0	15.57	18.47
	100000	9.1	0.08	0.18	10/0/0	49.19	57.92
100	20000	19.0	0.47	2.39	10/0/0	7.19	8.44
	50000	17.6	0.48	1.29	10/0/0	20.74	24.86
	100000	17.3	0.97	2.79	10/0/0	61.74	66.85
200	20000	35.5	12.83	15.79	3/0/7	14.54	18.41
	50000	35.1	67.16	100.31	5/0/5	38.59	44.85
	100000	34.8	196.36	397.02	7/0/3	87.97	96.52

Table 8: Parallel algorithms on instances of Class 7.

8 Conclusions

We considered a variant of the knapsack problem with setups associated to classes of items. We studied alternative ILP formulations and analysed their properties in terms of linear programming relaxation. We proposed a generic Branch-and-Bound framework capable of embedding different relaxations and we showed how to solve these relaxations via new combinatorial algorithms (one of which is based on Column Generation). Finally, we proposed a parallel algorithm called FMT which combines the strengths of the new Branch-and-Bound algorithms and of an improved Dynamic Programming algorithm. We computationally compared the performances of the state-of-the-art algorithms for KPS with FMT. The outcome of these experiments is that FMT is capable of efficiently solving all the instances of the literature and it is the best algorithm for instances with a small number of classes.

Future lines of research An important generalization of KPS arises when lower and upper bounds are imposed on the total weight of the selected items for each class (if used). While the case where an upper bound is imposed has been studied by [19], to the best of our knowledge the case with a lower bound has not been considered so far in the literature. A challenging topic in this area is thus the extension of our approaches to these new constraints, that may prevent the linear-time algorithms developed for LP1 and LP3 from being valid. In a similar way, the interaction of setup costs and different constraints, e.g., as precedences and/or incompatibilities among items, may be worth of studying.

Acknowledgments

The authors thank Federico Della Croce, Fabio Salassa and Rosario Scatamacchia for making the source code for their algorithms available. Thanks are also due to three anonymous referees for helpful comments. The research of Michele Monaci has been supported by Air Force Office of Scientific Research, grant FA9550-17-1-0067.

References

References

- [1] U. Akinc. Approximate and exact algorithms for the fixed-charge knapsack problem. *European Journal of Operational Research*, 170(2):363–375, 2006.
- [2] N. Altay, P. R. Jr., and K. Bretthauer. Exact and heuristic solution approaches for the mixed integer setup knapsack problem. *European Journal of Operational Research*, 190(3):598–609, 2008.
- [3] E. Balas and E. Zemel. An algorithm for large zero-one knapsack problems. *Operations Research*, 28(5):1130–1154, 1980.
- [4] E. Chajakis and M. Guignard. Exact algorithms for the setup knapsack problem. *INFOR: Information Systems and Operational Research*, 32(3):124–142, 1994.
- [5] K. Chebil and M. Khemakhem. A dynamic programming algorithm for the knapsack problem with setup. *Computers & Operations Research*, 64:40–50, 2015.
- [6] F. Della Croce, F. Salassa, and R. Scatamacchia. An exact approach for the 0–1 knapsack problem with setups. *Computers & Operations Research*, 80:61–67, 2017.
- [7] J. Desrosiers and M. Lübbecke. *Column Generation*, chapter A Primer in Column Generation. Springer US, Boston, MA, 2005.
- [8] M. Dyer. An $o(n)$ algorithm for the multiple-choice knapsack linear program. *Mathematical Programming*, 29(1):57–63, 1984.
- [9] M. Fischetti, A. Lodi, M. Monaci, D. Salvagnin, and A. Tramontani. Improving branch-and-cut performance by random sampling. *Mathematical Programming Computation*, 8:113–132, 2016.
- [10] M. Guignard. Solving makespan minimization problems with lagrangean decomposition. *Discrete Applied Mathematics*, 42(1):17–29, 1993.
- [11] E. Horowitz and S. Sahni. Computing partitions with applications to the knapsack problem. *Journal of ACM*, 21(2):277–292, 1974.
- [12] E. Johnson and M. Padberg. A note of the knapsack problem with special ordered sets. *Operations Research Letters*, 1(1):18–22, 1981.

- [13] H. Kellerer, U. Pferschy, and D. Pisinger. *Knapsack Problems*. Springer, Berlin Heidelberg, 2004.
- [14] M. Khemakhem and K. Chebil. A tree search based combination heuristic for the knapsack problem with setup. *Computers & Industrial Engineering*, 99:280–286, 2016.
- [15] E. Lin. A bibliographical survey on some well-known non-standard knapsack problems. *INFOR*, 36(4):274–317, 1998.
- [16] S. Martello and P. Toth. An upper bound for the zero-one knapsack problem and a branch and bound algorithm. *European Journal of Operational Research*, 1(3):169–175, 1977.
- [17] S. Martello and P. Toth. *Knapsack Problems: Algorithms and Computer Implementations*. John Wiley & Sons, Chichester, New York, 1990.
- [18] S. Martello, D. Pisinger, and P. Toth. Dynamic programming and strong bounds for the 0-1 knapsack problem. *Management Science*, 45(3):414–424, 1999.
- [19] S. Michel, N. Perrot, and F. Vanderbeck. Knapsack problems with setups. *European Journal of Operational Research*, 196(3):909–918, 2009.
- [20] U. Pferschy. Dynamic programming revisited: Improving knapsack algorithms. *Computing*, 63(4):419–430, 1999.
- [21] U. Pferschy and R. Scatamacchia. Improved dynamic programming and approximation results for the knapsack problem with setups. *International Transactions in Operational Research*, (forthcoming). . doi:[10.1111/itor.12381](https://doi.org/10.1111/itor.12381).
- [22] Y. Yang and R. Bulfin. An exact algorithm for the knapsack problem with setup. *International Journal of Operational Research*, 5(3):280–291, 2009.
- [23] E. Zemel. An $o(n)$ algorithm for the linear multiple choice knapsack problem and related problems. *Information Processing Letters*, 18(3):123–128, 1984.